

Essay

Word Embeddings

Nowadays, word embeddings are one of the main components of interactions between the Machine and the User. Most of Large Language Models (LLMs) would not be able to work if the model had to process the words of the user query directly. Word embeddings allow to transform those words into vectors while keeping the semantic meanings of the words. In this essay, I will explain all that I understood during this month of research on this subject.

First let's start with the problem statement, traditionally, a computer cannot understand the meaning of words and thus it is not able to find similar meanings in different words. We used Bag-of-Words (BoW) where each word was transformed into a new dimension. It leads to a semantic gap, in fact each word was orthogonal to others and thus computers were not able to calculate the similarity between two words. In summary, words were treated independently and the model missed the context linked to each word. This approach also created very sparse matrices because most of the dimensions were filled with zeros (One-hot encoding). As the vocabulary increased, the number of dimensions increased too, making the representation inefficient and unable to capture the relationships between words. [*Efficient Estimation of Word Representations in Vector Space*, Mikolov and al, 2013].

One-hot encoding

		cat	mat	on	sat	the
the =>		0	0	0	0	1
cat =>		1	0	0	0	0
sat =>		0	0	0	1	0
...						

One-hot encoding

From : tensorflow.org/text/guide/word_embeddings

“You shall know a word by the company it keeps” (John Rupert Firth, 1957). In other words, a word should never be separated from its context. So, to be sure to encode the meaning of a word into vectors, we need to catch its context in each sentence it is used in the training corpus and add this contextual information into the vector of the word. This idea is one of the foundations of modern Natural Language Processing.

To do so, we moved from One-Hot encoding to Dense Vectors where the information of the words is distributed in all the dimensions. In One-Hot encoding, two words are always independent because their vectors never share common values. Dense vectors solve this problem because each

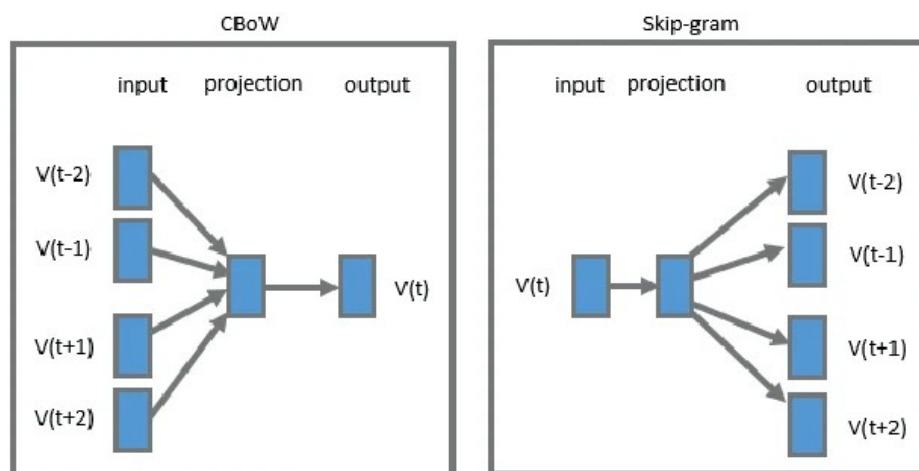
dimension contains part of the semantic information of the word. To create those vectors, we take the value of the target word but also each word in the same sentence as the word we want to vectorize. The vectors are generally much smaller than the vocabulary size, for example 100, 200 or 300 dimensions instead of tens of thousands. Even with fewer dimensions, they contain more semantic information.

To vectorize words there are two main algorithms: Skip-Gram and Continuous Bag-of-Words (CBOW). CBOW predicts a word from its context while Skip-Gram predicts the context from a word.

CBOW can be seen as a continuation of the Bag-of-Words explained before, but where we try to solve the sparse matrix problem. We start by encoding with One-Hot all the context words. The matrix obtained is multiplied by a matrix of weights W and the result gives vectors in the embedding dimensions. Finally, we calculate the mean of all embedding vectors of the context and thanks to that we are able to calculate the probability of the target word if this context is present. In practice, CBOW is fast to train and works well on frequent words because it averages the information from the context.

Skip-Gram follows the opposite logic: “If I know this word, what are the probable words next to it?”. We One-Hot encode only the target word and multiply it by the weight matrix. Then the algorithm tries to maximize the likelihood of the words appearing next to another. Instead of predicting one target from several context words, it predicts several context words from one target word. This method generally gives better semantic representations because the model learns more precisely the relationships between words.

In practice, Skip-Gram is often preferred since if we use CBOW on rare words they will be diluted inside the average context vectors, while with Skip-Gram the model will learn the links associated with this rare word. This is very important in real-world corpora because some technical or domain-specific words may appear only a few times but still carry important meanings.



Word2Vec (CBOW and Skip-gram)

From : [Sentiment analysis on Twitter data with semi-supervised Doc2Vec](#), Bilgin, 2017

The weight matrix is thus the important part of the algorithm. To visualize it more intuitively, it is like a lookup table where you give the line and it gives the vector representation associated with the

word. The weight matrix is learned based on the same principle as Neural Networks and backpropagation. We give input data, calculate the loss function with the output and the expected output, and backpropagate small changes to the weights. However, differently from usual neural networks, there is generally no hidden activation step since the goal is not deep learning and non-linear classification, but semantic representation learning (there is a softmax though). The loss function used is generally the cross-entropy loss function computed from the softmax of the output probabilities.

When all the words are vectorized into the same dimensions, we can now make vector arithmetic. Since they encode their meanings and the meaning of their contexts, we can calculate the similarity between two vectors by calculating the angle between them using cosine similarity. But we can also make operations like the famous:

King-Man+Woman $\approx$ Queen

This operation is possible because the difference Woman – Man encodes the information related to gender. Thus, if we add this difference to the vector King, we obtain the vector corresponding to the female equivalent of King, i.e. Queen. This example shows that embeddings do not only memorize words but also semantic relationships between concepts.



From : jalamar.github.io/illustrated-word2vec/

Description : Instead of using number we used color, the more it is red the closer it is from 1 and the more it is blue the closer from 0. The resulting vector from "king-man+woman" doesn't exactly equal "queen", but "queen" is the closest word to it from the 400,000 word embeddings we have in this collection.

Now let's move to the practical applications of word embeddings. Word embedding models are generally not trained directly on whole words but on tokens. A token can be a word, a subword or even a punctuation symbol depending on the tokenizer used. This is particularly useful because many languages contain huge vocabularies and unknown words. By splitting words into smaller units, models are able to generalize better on unseen text.

In my lab project, word embeddings are used on the query of the students and the biography of the tutors. Since those paragraphs are written in human language, we need to preprocess them by letting them pass through a preprocessing pipeline.

Step 1: Tokenization.

The text is separated into tokens that can be processed by the model.

Step 2: Stopword removal.

Those words have very few meanings and are of no use in our context, so we remove them. Words such as “the”, “is” or “and” appear frequently but generally carry little semantic information for matching students with tutors.

Sometimes additional preprocessing steps are also added, such as lowercasing, punctuation cleaning or stemming. The goal is to reduce noise and improve the quality of the embeddings.

After we have vectorized each word of the query, we need to create one vector to summarize the whole profile of the user. To do that, I used mean pooling, which consists of averaging all the embedding vectors of the sentence. Even if this approach is simple, it often gives good results in semantic similarity tasks because it captures the global meaning of the sentence.

After all of that we can compare vectors between each other with cosine similarity and find the K Nearest Neighbors of our query. This allows us to retrieve the tutors whose profiles are semantically the closest to the student request, even if they do not use exactly the same words.

Finally, word embeddings have many advantages but also some limitations. Their main advantage is their ability to understand synonyms and semantic similarity. They also allow models to handle unseen queries because similar meanings are represented by nearby vectors. Another important advantage is the multilingual potential, since embeddings from different languages can sometimes be aligned into the same semantic space.

However, embeddings also have disadvantages. One important issue is semantic noise. During my experiments, some unrelated words were sometimes considered similar because they appeared in similar contexts (e.g. Physics and Computer linked by quantum). Embeddings also lack hard logical boundaries. They are very efficient for approximating meanings but they do not truly understand logic or reasoning.

Despite these limitations, word embeddings marked a major transition in the history of Natural Language Processing. They created the foundations for modern Large Language Models and transformer architectures. Today, embeddings are used everywhere: search engines, recommendation systems, chatbots and tutoring systems.